TrashMap PH — System Overview & Developer Guide

Purpose. Onboard the next developer in one read. Each system function below states what it does, the user role that triggers it, every backing table/endpoint/file, and the exact place to extend or change behavior.

Companions. Setup commands → `docs/HOW TOs.md`. Architecture & day-by-day → `docs/TEAM_HANDOFF_CONTEXT.md`. Demo accounts → `docs/TEST_ACCOUNTS.md`. DB source of truth → `supabase/schema.sql`.

0. Stack at a Glance

Layer	Stack	Path
LGU Dashboard (web)	Next.js (App Router) + TypeScript + Tailwind + React-Leaflet	`src/`
Driver / Citizen App (mobile)	Flutter + Dart + supabase_flutter + flutter_map	`client_app/`
Backend	Supabase (Postgres + PostGIS + Realtime + Auth)	`supabase/schema.sql`
Routing engine	OpenRouteService (ORS) with OSRM fallback	`src/lib/ors-directions.ts`, `src/lib/route-optimizer.ts`

Three roles, enforced by RLS — `admin`, `driver`, `citizen` (column `app_user_profiles.role`). Service role (server) bypasses RLS for optimizer, materializer, and `ROUTE_OPS_SECRET`-gated routes.

1. System Functions (Expanded)

1.1 Collection Point Setting (Set by admin)

Admins maintain the master list of fixed pickup locations (homes, barangay halls, MRFs). Every weekly route plans across this list, and every citizen sees their nearest points on the **Community Waste Map**.

Concern	Where
Table	`public.collection_points` (lat/lng, label, zone, `is_active`)

Concern	Where
API (CRUD)	`src/app/api/collection-points/route.ts`, `src/app/api/collection-points/[id]/route.ts`
Admin UI	Map click "Add point" + sidebar list in `src/components/layout/dashboard-shell.tsx`
RLS	`select` public on `is_active=true`; `insert/update` for `authenticated`; `delete` admin only

Extending it: add a new column → 1) edit `schema.sql` (`alter table collection_points add column …`), 2) extend Zod payload + insert in the POST handler, 3) surface field in the dashboard sidebar form, 4) re-run schema in Supabase SQL editor.

1.2 Weekly Route Creation (Set by admin for drivers to collect trash in collection points)

Admins draw an ordered weekly **template** (Mon–Sun + start/end hour) by clicking collection points in sequence. The system stores the template once; on its scheduled day the server **materializes** it into a dated `routes` row with optimized geometry, ETAs, and turn-by-turn steps when a driver hits Start.

Concern	Where
Tables	`route_templates`, `route_template_stops`, `routes`, `route_stops`
Create template	`POST /api/routes/templates` (`src/app/api/routes/templates/route.ts`)
Materialize → daily route	`src/lib/template-materialize.ts` (called from `start` endpoint)
Driver start	`POST /api/routes/templates/[id]/start` (`src/app/api/routes/templates/[id]/start/route.ts`)
Optimizer (ORS + fallback)	`src/lib/route-optimizer.ts`, `src/lib/ors-directions.ts`
Admin UI	Route Planner panel in `dashboard-shell.tsx` (Confirm Route modal)
Time gate (Manila TZ)	`src/lib/route-gate.ts` (mirrored mobile-side at `client_app/lib/utils/route_gate.dart`)

Extending it: new optimization heuristic → edit `route-optimizer.ts`. New gate behavior (e.g. allow late start window) → edit both `route-gate.ts` files and the test in `route-gate.test.ts`. New per-stop metadata → add column to `route_template_stops` + `route_stops` (materializer must copy it).

1.3 Reported Garbage Point (Citizens can report garbage points; 10+ reports become a Hotspot marked on the Map, where the dashboard operator can set a pickup point for it to clear the hotspot)

Citizens drop a pin from the mobile **Report** screen. Postgres trigger `trg_refresh_hotspots_on_reports` calls `refresh_hotspots_from_reports()` which DBSCAN-clusters recent unresolved reports (≥10 reporters in a 25 m epsilon) into `hotspots` rows. `refresh_risk_zones_from_hotspots()` then projects severity into `risk_zones` for the heat layer. The admin sees the hotspot on the dashboard map and "clears" it by adding a `collection_points` entry inside it (which then gets pulled into the next weekly route).

Concern	Where
Tables	`reports`, `hotspots`, `risk_zones`
Citizen submit	`client_app/lib/screens/report_screen.dart` → direct insert into `reports`
Cluster fn	`public.refresh_hotspots_from_reports()` — `schema.sql` (~line 62)
Risk zones fn	`public.refresh_risk_zones_from_hotspots()` — `schema.sql` (~line 466)
Trigger	`trg_refresh_hotspots_on_reports` (statement-level, AFTER INSERT/UPDATE/DELETE on `reports`)
Dashboard render	Heat layer + hotspot circles in `src/components/map/lgu-map.tsx` + Realtime subscriptions in `dashboard-shell.tsx`
Mobile render	`client_app/lib/screens/map_screen.dart`
RLS	`hotspots` + `risk_zones` public read; `service_role` ALL policy lets the trigger rewrite both tables

Extending it: to change the hotspot threshold, edit constants in `refresh_hotspots_from_reports` (`minpoints := 10`, `eps := 25`). For a missed-pickup auto-report variant, see `src/lib/route-end.ts` (`buildMissedPickupReports`).

1.4 Collection Schedule on Mobile App for Citizens

Citizens see upcoming pickups in their zone (day of week + time window) so they know when to bring out trash. Today the schedule view reads from the `schedules` table; eventually it will read directly from `route_templates` for a single source of truth.

Concern	Where
Tables	`schedules` (legacy citizen-facing), `route_templates` (admin-facing source)
Mobile UI	`client_app/lib/screens/schedule_screen.dart`, `home_shell.dart`
RLS	`schedules` public read

Extending it: to merge schedules into templates, query `route_templates` joined with `route_template_stops` filtered by the citizen's zone (or nearest collection points). Add an active-only filter (`is_active = true`) and order by `recurrence_day` then `start_hour`.

1.5 Community Waste Map (Citizens see pin points of Reported Garbage Points and Main Collection points in their zone)

Single map for citizens that overlays four layers: their own and neighbors' **reports**, the official **collection points**, the live **hotspots**, and an optional **risk zones** heatmap. Realtime channels keep all four layers in sync without refresh.

Concern	Where
Mobile map	`client_app/lib/screens/map_screen.dart` (uses `flutter_map`)
Web map	`src/components/map/lgu-map.tsx` (admin variant)
Realtime channels	`reports`, `hotspots`, `risk_zones`, `truck_pings` (publication `supabase_realtime` in `schema.sql`)
RLS	All four tables: public `select`; `reports` insert by `authenticated`

Extending it: add a new layer (e.g. recyclers) → 1) ensure RLS allows citizen read, 2) add Supabase channel subscription, 3) draw the `MarkerLayer`/`PolygonLayer`. Keep all map heavy work behind a memoized list to avoid the rebuild storm bug we hit in `navigation_screen.dart` (see comments in that file).

1.6 Driver Assigning (Operator can choose multiple drivers to set to a Weekly route)

Admins assign one or more drivers to a weekly **template** (not a daily route). Each driver then sees that template in their mobile app and can start it on its scheduled day. Assignments are soft-deactivated (`is_active=false`) so historical audit stays intact.

Concern	Where
Tables	`route_template_assignments` (perma assign), `route_assignments` (per-day, created on materialize)
Assign API	`POST /api/routes/templates/[id]/assignments` (`src/app/api/routes/templates/[id]/assignments/route.ts`)
Unassign	`DELETE /api/routes/templates/[id]/assignments/[assignmentId]/route.ts` (sets `is_active=false`)
Admin UI	Driver Assignment panel in `dashboard-shell.tsx` (multi-select + ops token)
Driver app sees	`client_app/lib/services/api_client.dart` → `getMyAssignedTemplates()`
Auth	`ROUTE_OPS_SECRET` header (`x-route-ops-secret`) — not user JWT; see `src/lib/route-ops.ts` `isRouteOpsAuthorized`

Extending it: to support driver swap mid-day, deactivate the active `route_assignments` row and insert a new one with the new `driver_id`. To add a "primary driver" flag, add a boolean column on `route_template_assignments` and surface it in the assignment payload.

2. Bonus Functions (Beyond the User's Six)

2.1 Driver Navigation HUD + Live Telemetry (Phase 6)

When a driver taps Start, the mobile app opens a turn-by-turn HUD: top instruction card, map with truck arrow, and a bottom sheet of stops with Confirm/Skip. GPS samples every 5 m flow into a 5 s telemetry POST loop, with offline buffering up to 240 fixes. Within 50 m of a stop for 3 s the engine flips that stop to `arrived` and writes `route_progress`.

Concern	Where
Screen	`client_app/lib/screens/navigation_screen.dart`
Pure logic (testable)	`client_app/lib/services/step_engine.dart`
GPS + queue	`client_app/lib/services/telemetry_service.dart`
Telemetry endpoint	`POST /api/routes/[routeId]/telemetry`

Concern	Where
End route	`POST /api/routes/[routeId]/end` (creates `report_type='missed_pickup'` for unresolved)
Admin live map	`LiveTruckMarkers` in `src/components/map/lgu-map.tsx` (Realtime on `truck_pings`)
Admin playback	`src/components/dashboard/route-report-modal.tsx` + `GET /api/routes/[routeId]/report`

Critical patterns to preserve when editing `navigation_screen.dart`:

- `_disposed` guard on every async resume (`if (_disposed) return;`).
- Throttle `setState` to ~1/800 ms — full-tree rebuild storms were the #1 ANR source.
- `_autoFollow` toggles off on user pan, on via FAB tap. Never `_mapController.move()` unconditionally.
- Always include `truck_id` on `route_progress` inserts (NOT NULL).

2.2 Route Optimizer + Cron

`POST /api/optimize-routes` (manual, admin) and `POST /api/optimize-routes/scheduled` (`OPTIMIZER_CRON_SECRET`) call `route-optimizer.ts` to compute distance/ETA and persist polylines. Failure auto-falls back to `mockEstimate` so dashboards never go blank.

2.3 Demo Seeding

`POST /api/demo/day3-seed` and `day4-seed` (gated by `DEMO_SEED_SECRET`) populate deterministic fixtures. Useful when you wipe Supabase between sprints.

2.4 Audit + Notifications

`route_audit_logs` (every state transition) and `route_notifications_log` (citizen + admin push surface) are appended via helpers in `src/lib/route-ops.ts` (`appendRouteAudit`, `appendRouteNotification`). Add new event types here to keep the timeline complete.

3. Module Map (where to look first)

```
trashmap-ph/
■■ supabase/schema.sql                ← single source of truth for DB
■■ src/
■  ■■ app/api/                        ← every Next.js endpoint
■  ■  ■■ routes/templates/...         ← weekly template CRUD + assign + start
```

```
■  ■  ■■ routes/[routeId]/              ← daily route ops: start, end, telemetry, repo
rt, arriving, stops
■  ■  ■■ collection-points/             ← admin pin CRUD
■  ■  ■■ optimize-routes/               ← optimizer (manual + scheduled)
■  ■  ■■ demo/                          ← seed endpoints
■  ■■ components/
■  ■  ■■ layout/dashboard-shell.tsx     ← the entire admin SPA
■  ■  ■■ dashboard/route-report-modal.tsx ← post-route playback
■  ■  ■■ map/lgu-map.tsx                ← admin map (reports, hotspots, trucks, polyli
nes)
■  ■■ lib/
■     ■■ route-ops.ts                   ← auth helpers, audit, notifications
■     ■■ route-optimizer.ts             ← ORS + fallback
■     ■■ ors-directions.ts              ← geometry + turn-by-turn
■     ■■ template-materialize.ts        ← template → daily route
■     ■■ route-gate.ts / .test.ts       ← Manila-TZ start window
■     ■■ route-end.ts / .test.ts        ← missed-pickup builder + status
■■ client_app/
   ■■ lib/screens/                     ← all Flutter screens
   ■  ■■ navigation_screen.dart        ← driver HUD (most complex)
   ■  ■■ template_preview_screen.dart  ← pre-start preview
   ■  ■■ assigned_templates_screen.dart ← driver home
   ■  ■■ report_screen.dart, map_screen.dart, schedule_screen.dart  ← citizen
   ■■ lib/services/
   ■  ■■ api_client.dart               ← typed Dio wrapper
   ■  ■■ step_engine.dart              ← pure nav logic (unit-tested)
   ■  ■■ telemetry_service.dart        ← GPS + offline queue
   ■■ lib/utils/route_gate.dart        ← mirror of server gate
```

4. How to Make a Change Safely

Change type	Steps
New column on existing table	Edit `schema.sql` → re-run in SQL editor → update Zod/Dart parsers → update RLS if needed
New endpoint	Add `src/app/api/<path>/route.ts`, reuse helpers in `src/lib/route-ops.ts`, check auth (`getBearerUserId` or `isRouteOpsAuthorized`), return `{ ok, ... }` shape consistent with `api_client.dart`
New driver UI step	Update `step_engine.dart` (pure logic) + add unit test in `client_app/test/step_engine_test.dart`, then wire into `navigation_screen.dart`
New admin panel	Append a new section to `dashboard-shell.tsx` and load via existing Supabase client; respect RLS (admins only)

Change type	Steps
New realtime layer	Add `alter publication supabase_realtime add table <name>;` in `schema.sql`, subscribe in dashboard or app
Trigger logic touches RLS-enabled table	Confirm a `service_role` policy exists (`hotspots_all_service`, `risk_zones_all_service` are precedents)

5. Test, Lint, Run

```
# Web
npm install
npm run dev          # http://localhost:3000
npm run lint
npm run test         # vitest (route-end, route-gate)
npx tsc --noEmit

# Mobile (replace path / project / key with yours)
cd client_app
flutter pub get
flutter analyze
flutter test
flutter run -d emulator-5554 ^
  --dart-define=SUPABASE_URL="https://YOUR_PROJECT.supabase.co" ^
  --dart-define=SUPABASE_ANON_KEY="YOUR_ANON_KEY" ^
  --dart-define=API_BASE_URL="http://10.0.2.2:3000"
```

End-to-end smoke test recipe → `docs/HOW TOs.md` HOW TO 10 (Day 3) and HOW TO 21 (Phase 6 nav).

6. Common Pitfalls (Solved Once — Don't Re-Break)

Pitfall	Symptom	Fix in tree
`setState` after dispose	App crash on second route start	`_disposed` guard in `navigation_screen.dart`
Rebuild storm on every GPS fix	ANR ("isn't responding")	800 ms throttle + `_autoFollow` flag
Auto-follow blocks user pan	User can't scroll map	`onPositionChanged.hasGesture` toggles `_autoFollow=false`
`route_progress` NOT NULL `truck_id`	"null value in column truck_id"	Fetch route truck once, include in every insert
Trigger DELETE on RLS table	"DELETE requires a WHERE clause"	`service_role` ALL policies on `hotspots`, `risk_zones`

Pitfall	Symptom	Fix in tree
Duplicate route per template/day	`PostgrestException 406` from `.maybeSingle()`	Unique partial index `idx_routes_template_date` + dedupe `do $$ block` in `schema.sql`
ESLint scanning Flutter build	Lint errors in `wakelock_plus/no_sleep.js`	`client_app/**` in `globalIgnores` (`eslint.config.mjs`)

7. Where to Ask Yourself Before Editing

1. **Which role triggers this code path?** Admin / driver / citizen / cron / service.
2. **Which RLS table am I writing to?** Pick the right Supabase client (anon for citizen JWT, service for server tasks).
3. **Is there a mobile-side mirror?** Route gate, NavStop status enum, telemetry payload — keep both sides in lockstep.
4. **Did I add a Realtime channel?** If the admin map should reflect it live, add to publication.
5. **Did I add a unit test?** `route-end`, `route-gate`, and `step_engine` are templates — copy their style.

8. Glossary

- **Template** — recurring weekly route definition (`route_templates`).
- **Route** — concrete instance for a specific date (`routes`), materialized from a template.
- **Stop** — single pickup location on a route (`route_stops`); status = `pending | arrived | completed | skipped | missed`.
- **Progress** — driver-side ledger of stop outcomes (`route_progress`).
- **Ping** — single GPS sample written by the driver app (`truck_pings`).
- **Hotspot** — clustered citizen reports (`hotspots`); ≥10 reporters in 25 m.
- **Risk zone** — derived heat polygon from hotspots (`risk_zones`).
- **Gate** — Manila-TZ time window check (early / on-time / late).
- **Ops token** — `ROUTE_OPS_SECRET` HTTP header bypassing user JWT for admin route ops.

Last updated: Phase 7 (driver nav stable, RLS service-role policies for trigger writebacks).